

IMY 220 Assignment 1

Node.js, npm and Socket.IO

Scenario

You have joined a development team working on a small real-time messenger application.

Another developer started building the application, but the project contains several bugs and incomplete functionality. The supplied code may prevent the application from working correctly.

You are required to inspect the existing codebase, repair the application and complete the messenger functionality.

This is not a visual design assignment. The supplied HTML and CSS should not be substantially changed unless instructed.

Learning outcomes

After completing this assignment, students should be able to:

- work with an existing Node.js codebase;
 - manage a project using npm;
 - configure Express and an HTTP server;
 - implement real-time communication using Socket.IO;
 - validate data received by a server;
 - debug existing code;
 - track and communicate application state across connected clients;
 - explain basic technical decisions.
-

Supplied starter project

The supplied project contains:

- a basic HTML interface;
- CSS styling;
- partially implemented server-side JavaScript;
- partially implemented client-side JavaScript;
- a partially configured package.json file;

- an `answers.txt` file.

Your task is to repair and complete the existing project rather than creating a new application from scratch.

Where possible, repair the existing implementation rather than replacing it completely.

Section 1: npm and project setup — 5 marks

Inspect and correct the supplied `package.json` file.

Before running the application, install the required dependencies using npm.

Students must:

1. Correct the project name and description. **(1)**
 2. Add the student's details to the author field. **(1)**
 3. Add an npm script named `start` that runs `server.js`. **(1)**
 4. Install the packages required by the application. **(1)**
 5. Ensure the application can be started using: `npm start` **(1)**
-

Section 2: Adapt the supplied server — 5 marks

The supplied server contains errors and incomplete functionality.

Repair the server so that:

1. An Express application and HTTP server are configured correctly; **(1)**
 2. The files in the public folder are served correctly; **(1)**
 3. The Socket.IO server is attached correctly; **(1)**
 4. The server uses `process.env.PORT || 3000`; **(1)**
 5. The HTTP server starts listening and displays: Messenger server running on `http://localhost:<port>` **(1)**
-

Section 3: User connections — 2 marks

Complete the client-side connection functionality.

When a client connects:

- Log the client's socket ID in the browser console. (1)
 - Display system messages received from the server in the interface. (1)
-

Section 4: Send and receive messages — 17 marks

Complete the messenger functionality.

Each accepted message must contain:

```
{  
  username: "Sarah",  
  text: "Hello everyone",  
  timestamp: "2026-08-03T08:30:00.000Z"  
}
```

Students must:

1. Remove or replace the supplied "hello" functionality. (1)
2. Prevent the form from refreshing. (1)
3. Read the username and message text. (2)
4. Emit a message event containing both values. (2)
5. Listen for the event on the server. (2)
6. Broadcast the complete message object. (2)
7. Display the sender and text. (2)
8. Create the timestamp. (2)
9. Display the timestamp. (1)
10. Clear the message input after sending. (1)
11. Return focus to the message input. (1)

Example displayed message

08:30 - Sarah: Hello everyone

The timestamp may be displayed in any clear, readable format.

Section 5: Validation — 4 marks

The application must not accept:

- an empty display name;
- a display name containing only whitespace;
- an empty message;
- a message containing only whitespace.

Students must:

1. Validate the username and message on the server before broadcasting the message. **(2)**
2. Send an appropriate error event to the client when validation fails. **(1)**
3. Display the error in the supplied error element. **(1)**

The invalid message must not be broadcast to other connected users.

Section 6: Connected users — 3 marks

Display the number of users currently connected to the messenger.

Students must:

1. Determine the number of currently connected Socket.IO clients on the server. **(1)**
2. Send the updated total whenever a client connects or disconnects. **(1)**
3. Display the updated number of connected users in the interface. **(1)**

Example:

Users online: 2

Hint: Investigate whether Socket.IO provides a built-in way to determine the number of connected clients.

Section 7: Understanding — 4 marks

Answer the following questions in the supplied `answers.txt` file.

Question 1 — 1 mark

Why should the `node_modules` folder not be submitted?

Question 2 — 1 mark

What is the purpose of the `start` script in `package.json`?

Question 3 — 1 mark

Explain the difference between:

`socket.emit()`

and:

`io.emit()`

Question 4 — 1 mark

Why should username and message validation be performed on the server, even if validation is also performed in the browser?

Mark allocation

Section	Marks
npm and project setup	5
Adapt the supplied server	5
User connections	2
Send and receive messages	17
Validation	4
Connected users	3
Understanding	4
Total	40

Submission requirements

Submit the complete project folder, excluding the `node_modules` folder.

The submitted project must include:

- `package.json`;
- `package-lock.json`;
- `server.js`;
- the complete `public` folder;
- `answers.txt`.

The application must run after executing:

`npm install`

`npm start`